

Destructuring, Rest and Spread

The pattern on the left mirrors the shape on the right. Three dots collect on the left, and expand on the right.

Object destructuring: matched by NAME

```
const user = { id: 7, name: 'Ada', city: 'Bath', tz: 'GMT' };
```

```
const { name, city: town, role = 'guest', ...rest } = user;
```

name = 'Ada' → same key, plucked out
town = 'Bath' → renamed with a colon
role = 'guest' → default, the key was missing
rest = { id: 7, tz: 'GMT' } → everything left over

Array destructuring: matched by POSITION

```
const scores = [90, 80, 70, 60];
```

```
const [first, , third, ...others] = scores;
```

first = 90 → position 0
(the empty slot skips position 1)
third = 70 → position 2
others = [60] → rest must come LAST

Same three dots, opposite jobs

REST: on the left of the equals
Gathers what is left into one new value.

```
const [head, ...tail] = list;  
function sum(...numbers) { }
```

SPREAD: on the right, or in a call
Unpacks one value into many.

```
const copy = { ...user, city: 'Leeds' };  
const merged = [...a, ...b];
```

Spreading copies only ONE level deep: a nested object is still shared with the original. Later keys win, which is how { ...defaults, ...options } works. And destructuring a function parameter, function f({ id, name }), gives named arguments.